

## COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 4

---

**Name:****Student ID:****Section:**

1. Consider a sequence of memory address references given below. In the sequence, each word address is provided in both the decimal and binary formats. Below each address, the relative time at which these references occur is also listed. Memory contents and addresses are shown in the second table.

Data Access Sequence

| Time | Address Decimal | Address Binary |
|------|-----------------|----------------|
| 1    | 28              | 00 01 11 00    |
| 2    | 40              | 00 10 10 00    |
| 3    | 36              | 00 10 01 00    |
| 4    | 16              | 00 01 00 00    |
| 5    | 52              | 00 11 01 00    |
| 6    | 36              | 00 10 01 00    |
| 7    | 8               | 00 00 10 00    |
| 8    | 12              | 00 00 11 00    |
| 9    | 36              | 00 10 01 00    |
| 10   | 40              | 00 10 10 00    |
| 11   | 36              | 00 10 01 00    |
| 12   | 44              | 00 10 11 00    |
| 13   | 52              | 00 11 01 00    |
| 14   | 60              | 00 11 11 00    |
| 15   | 56              | 00 11 10 00    |
| 16   | 16              | 00 01 00 00    |
| 17   | 20              | 00 01 01 00    |
| 18   | 12              | 00 00 11 00    |
| 19   | 36              | 00 10 01 00    |
| 20   | 52              | 00 11 01 00    |

Memory

| Address Decimal | Address Binary | Data |
|-----------------|----------------|------|
| 8               | 00 00 10 00    | 345  |
| 12              | 00 00 11 00    | 456  |
| 16              | 00 01 00 00    | 4    |
| 20              | 00 01 01 00    | 831  |
| 24              | 00 01 10 00    | 546  |
| 28              | 00 01 11 00    | 65   |
| 32              | 00 10 00 00    | 7    |
| 36              | 00 10 01 00    | 435  |
| 40              | 00 10 10 00    | 9    |
| 44              | 00 10 11 00    | 23   |
| 48              | 00 11 00 00    | 234  |
| 52              | 00 11 01 00    | 97   |
| 56              | 00 11 10 00    | 19   |
| 60              | 00 11 11 00    | 1000 |

## COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 4

---

For **fully-associative cache**: a word can be stored in any block

Assume that the least-recently used (LRU) scheme is used where appropriate, where the least recently inserted or accessed element is replaced on conflict. Also, when inserting an element into the cache, if there are multiple empty slots for one index, you should insert the new element into the left-most slot (first available slot).

- a) Use the direct-mapped cache to facilitate memory access for the memory sequence above. You should fill in the binary form of the Tag values. Show the final contents of the cache in the table below, and compute the hit rate and miss rate.

| Index | Cache |     |      |
|-------|-------|-----|------|
|       | V     | Tag | Data |
| 0     |       |     |      |
| 1     |       |     |      |
| 2     |       |     |      |
| 3     |       |     |      |
| 4     |       |     |      |
| 5     |       |     |      |
| 6     |       |     |      |
| 7     |       |     |      |

Direct-Mapped Cache Summary

**Hit Rate:**

**Miss Rate:**

- b) Use the 2-way set associative cache to facilitate memory access for the memory sequence above. You should fill in the binary form of the Tag values. Show the final contents of the cache in the table below, and compute the hit rate and miss rate.

## COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 4

---

| Set | 2-way Set Associative Cache |     |      |   |     |      |
|-----|-----------------------------|-----|------|---|-----|------|
|     | V                           | Tag | Data | V | Tag | Data |
| 0   |                             |     |      |   |     |      |
| 1   |                             |     |      |   |     |      |
| 2   |                             |     |      |   |     |      |
| 3   |                             |     |      |   |     |      |

2-way Set Associative Cache Summary

**Hit Rate:**

**Miss Rate:**

- c) Use the 4-way set associative cache to facilitate memory access for the memory sequence above. You should fill in the binary form of the Tag values. Show the final contents of the cache in the table below, and compute the hit rate and miss rate.

| Set | 4-way Set Associative Cache |     |      |   |     |      |   |     |      |   |     |      |
|-----|-----------------------------|-----|------|---|-----|------|---|-----|------|---|-----|------|
|     | V                           | Tag | Data | V | Tag | Data | V | Tag | Data | V | Tag | Data |
| 0   |                             |     |      |   |     |      |   |     |      |   |     |      |
| 1   |                             |     |      |   |     |      |   |     |      |   |     |      |

4-way Set Associative Cache Summary

**Hit Rate:**

**Miss Rate:**

- d) Use the fully-associative cache to facilitate memory access for the memory sequence above. You should fill in the binary form of the Tag values. Show the final contents of the cache in the table below, and compute the hit rate and miss rate.

**COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE  
ASSIGNMENT 4**

---

Fully Associative Cache

| V | Tag | Data |
|---|-----|------|
|   |     |      |
|   |     |      |
|   |     |      |
|   |     |      |
|   |     |      |
|   |     |      |
|   |     |      |
|   |     |      |

Fully Associative Cache Summary

**Hit Rate:**

**Miss Rate:**

2. Fill in the following table for different caches. Assume all caches are direct mapped.

| Address<br>Bits | Cache<br>Size | Block<br>Size | Tag<br>Bits | Index<br>Bits | Offset<br>Bits | Bits per<br>Row |
|-----------------|---------------|---------------|-------------|---------------|----------------|-----------------|
| 16              | 4KB           | 4B            |             |               |                |                 |
| 16              | 16KB          | 8B            |             |               |                |                 |
| 32              | 8KB           | 8B            |             |               |                |                 |
| 32              | 32KB          | 16B           |             |               |                |                 |
| 32              | 64KB          |               | 16          | 12            | 4              | 145             |
| 32              | 512KB         |               |             |               | 5              |                 |
| 64              |               | 64B           |             | 14            |                |                 |
| 64              | 2048KB        |               | 43          |               | 7              | 1068            |

**COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE  
ASSIGNMENT 4**

---

3. Assume a direct-mapped cache and data access sequence given below where the memory and its contents are shown in the table on the right. Memory is byte addressed and addresses are given in decimal. Assume five-bit addresses and a one-byte data block.

| Data Access Sequence |         | I | Cache |      | Memory |         |      |
|----------------------|---------|---|-------|------|--------|---------|------|
| Op.                  | Address |   | Tag   | Data | Binary | Address | Data |
| read                 | 30      |   |       |      |        | 20      | 5    |
| read                 | 31      |   |       |      |        | 21      | 9    |
| read                 | 30      |   |       |      |        | 22      | 47   |
| read                 | 26      | 0 |       |      |        | 23      | 13   |
| read                 | 25      |   |       |      |        | 24      | 76   |
| read                 | 28      | 1 |       |      |        | 25      | 6    |
| read                 | 23      | 2 |       |      |        | 26      | 43   |
| read                 | 25      | 3 |       |      |        | 27      | 8    |
| read                 | 28      |   |       |      |        | 28      | 12   |
| read                 | 20      |   |       |      |        | 29      | 4    |
| read                 | 29      |   |       |      |        | 30      | 72   |
|                      |         |   |       |      |        | 31      | 101  |

hits =

hit rate =

misses =

miss rate =

4. Assume a direct-mapped cache and data access sequence given below where the memory and its contents are shown in the table on the right. Memory is byte addressed and addresses are given in decimal. Assume five-bit addresses and a two-byte data block.

## COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 4

Data Access Sequence

| Op.  | Address |
|------|---------|
| read | 30      |
| read | 31      |
| read | 30      |
| read | 26      |
| read | 25      |
| read | 28      |
| read | 23      |
| read | 25      |
| read | 28      |
| read | 20      |
| read | 29      |

I

Cache

|   | Tag | Data | Data |
|---|-----|------|------|
| 0 |     |      |      |
| 1 |     |      |      |
| 2 |     |      |      |
| 3 |     |      |      |

Memory

| Binary | Address | Data |
|--------|---------|------|
|        | 20      | 5    |
|        | 21      | 9    |
|        | 22      | 47   |
|        | 23      | 13   |
|        | 24      | 76   |
|        | 25      | 6    |
|        | 26      | 43   |
|        | 27      | 8    |
|        | 28      | 12   |
|        | 29      | 4    |
|        | 30      | 72   |
|        | 31      | 101  |

hits =

hit rate =

misses =

miss rate =

5. Assume a 2-way set associative cache and data access sequence given below where the memory and its contents are shown in the table on the right. Memory is byte addressed and addresses are given in decimal. Assume five-bit addresses and a two-byte data block. You should assume that the replacement policy used here is LRU (least recently used).

Data Access Sequence

| Op. | Address |
|-----|---------|
| lb  | 30      |
| lb  | 31      |
| lb  | 30      |
| lb  | 26      |
| lb  | 25      |
| lb  | 28      |
| lb  | 23      |
| lb  | 25      |
| lb  | 28      |
| lb  | 20      |
| lb  | 29      |

S

Cache

|   | Tag | Data | Data |
|---|-----|------|------|
| 0 |     |      |      |
| 1 |     |      |      |

Memory

| Binary | Address | Data |
|--------|---------|------|
|        | 20      | 5    |
|        | 21      | 9    |
|        | 22      | 47   |
|        | 23      | 13   |
|        | 24      | 76   |
|        | 25      | 6    |
|        | 26      | 43   |
|        | 27      | 8    |
|        | 28      | 12   |
|        | 29      | 4    |
|        | 30      | 72   |
|        | 31      | 101  |

## COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 4

---

hits =

hit rate =

misses =

miss rate =

6. Consider a sequence of memory access references given below. In the sequence, each address is provided in the decimal format (dec) along with the operation (LDR or STR). Along with each address, the relative time at which these references occur is also listed. Memory contents and addresses are shown in the second table.

| Data Access Sequence |     |               |             | Memory        |               |             |
|----------------------|-----|---------------|-------------|---------------|---------------|-------------|
| Time                 | Op. | Address (dec) | Data(hex)   | Address (dec) | Address (bin) | Data (hex)  |
| 1                    | LDR | 8             |             | 8             | 00 00 10 00   | aa aa aa aa |
| 2                    | LDR | 52            |             | 12            | 00 00 11 00   | bb bb bb bb |
| 3                    | LDR | 12            |             | 16            | 00 01 00 00   | cc cc cc cc |
| 4                    | LDR | 20            |             | 20            | 00 01 01 00   | dd dd dd dd |
| 5                    | STR | 16            | db db db db | 24            | 00 01 10 00   | ee ee ee ee |
| 6                    | LDR | 44            |             | 28            | 00 01 11 00   | ff ff ff ff |
| 7                    | LDR | 40            |             | 32            | 00 10 00 00   | ca ca ca ca |
| 8                    | LDR | 32            |             | 36            | 00 10 01 00   | 1A 2B 3C 4D |
| 9                    | STR | 60            | 15 05 15 05 | 40            | 00 10 10 00   | 12 34 56 78 |
| 10                   | STR | 36            | 20 18 20 18 | 44            | 00 10 11 00   | 78 56 34 12 |
| 11                   | LDR | 8             |             | 48            | 00 11 00 00   | ab cd ef 00 |
| 12                   | LDR | 12            |             | 52            | 00 11 01 00   | ee 02 04 ee |
| 13                   | LDR | 40            |             | 56            | 00 11 10 00   | 00 00 00 00 |
| 14                   | LDR | 44            |             | 60            | 00 11 11 00   | 11 11 11 11 |
| 15                   | LDR | 48            |             | 64            | 01 00 00 00   | 0f 1e 2d 3c |
| 16                   | LDR | 52            |             |               |               |             |
| 17                   | STR | 40            | ca ca ca ca |               |               |             |
| 18                   | STR | 44            | db db db db |               |               |             |
| 19                   | STR | 8             | 11 22 33 44 |               |               |             |
| 20                   | STR | 12            | 55 66 77 88 |               |               |             |

Now consider two different 2-word caches shown on the next page. Assume that each of the two caches are used independently to facilitate memory access for the sequence above. For each cache type, assume that the cache is initially empty.

## COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 4

---

Assume that the least-recently used (LRU) scheme is used where appropriate, where the least recently inserted or accessed element is replaced on conflict. Also, when inserting an element into the cache, if there are multiple empty slots for one index, you should insert the new element into the first available slot.

For parts (a) and (b) assume the initial memory contents are the ones given above. Update memory in the space above for both parts (try to distinguish between the two if needed.)

- a) Use the **direct-mapped** cache shown below with a **block size of two words** to facilitate memory access for the access sequence above. For write hits and misses use the **write back** and **write allocate** policies respectively. You should fill in the binary form of the tag values and hex values for data. When you replace any content in the cache, simply put a cross sign ( **X** ) to overwrite a value, and add the new value in the available space. Show the final contents of the memory and cache below, and compute the hit and miss rates.

| Cache |   |   |     |      |      |
|-------|---|---|-----|------|------|
|       | D | V | Tag | Data | Data |
| 0     |   |   |     |      |      |
| 1     |   |   |     |      |      |
| 2     |   |   |     |      |      |
| 3     |   |   |     |      |      |

- b) Use the **2-way set associative** cache shown below with a of **block size two words** to facilitate memory access for the access sequence above. For write hits and misses use the **write through** and **write around** policies respectively. You should fill in the binary form of the tag values and hex values for data. When you replace any content in the cache, simply put a cross sign (x) to overwrite a value, and add the new value in the available space. Show the final contents of the memory and cache below, and compute the hit and miss rates.

## COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 4

| Cache |   |     |      |
|-------|---|-----|------|
|       | V | Tag | Data |
| 0     |   |     |      |
|       |   |     |      |
| 1     |   |     |      |
|       |   |     |      |

- c) Which of the above two caches performs better and why?
7. Show the final data and tags of the direct-mapped cache shown below for the given memory contents and data access sequence. As shown, you have 8-bit addresses and 4 bytes of data in each block. You can assume write-through and write-allocate write strategies on write hits and misses respectively.

| Data Access Sequence |             |      | Memory      |      |
|----------------------|-------------|------|-------------|------|
| Op.                  | Address     | Data | Address     | Data |
| lw                   | 00 01 11 00 |      | 00 00 10 00 | 435  |
| lw                   | 00 10 10 00 |      | 00 00 11 00 | 9    |
| sw                   | 00 10 01 00 | 32   | 00 01 00 00 | 47   |
| lw                   | 00 01 00 00 |      | 00 01 01 00 | 13   |
| sw                   | 00 11 01 00 | 642  | 00 01 10 00 | 7586 |
| lw                   | 00 10 10 00 |      | 00 01 11 00 | 86   |
| lw                   | 00 01 11 00 |      | 00 10 00 00 | 43   |
| sw                   | 00 10 01 00 | 124  | 00 10 01 00 | 48   |
| lw                   | 00 10 00 00 |      | 00 10 10 00 | 673  |
| lw                   | 00 10 10 00 |      | 00 10 11 00 | 4    |
|                      |             |      | 00 11 00 00 | 972  |
|                      |             |      | 00 11 01 00 | 1003 |

| I  |  | Direct-Mapped Cache |     |      |
|----|--|---------------------|-----|------|
|    |  | V                   | Tag | Data |
| 00 |  |                     |     |      |
| 01 |  |                     |     |      |
| 10 |  |                     |     |      |
| 11 |  |                     |     |      |

hits =

hit rate =

misses =

miss rate =

8. Fill the following, show final cache

**Block size =** \_\_\_\_\_

**N =**  
\_\_\_\_\_

**Assoc =**  
\_\_\_\_\_

**COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE  
ASSIGNMENT 4**

---

Memory

|    |     |
|----|-----|
| 20 | 7   |
| 21 | 3   |
| 22 | 27  |
| 23 | 32  |
| 24 | 101 |
| 25 | 78  |
| 26 | 59  |
| 27 | 24  |
| 28 | 56  |
| 29 | 87  |
| 30 | 36  |
| 31 | 98  |

0

1

Cache

| Address | Data |  |
|---------|------|--|
|         |      |  |
|         |      |  |
|         |      |  |
|         |      |  |
|         |      |  |
|         |      |  |
|         |      |  |
|         |      |  |
|         |      |  |

Processor

1. Read 24
2. Read 25
3. Read 26
4. Read 24
5. Read 21
6. Read 26
7. Read 24
8. Read 26
9. Read 27

hits =

hit rate =

misses =

miss rate =

9. Assume a **direct-mapped cache** of size **8KB**, each block having two 32-bit words. The system is based on 32-bit physical address and uses byte addressing. Find the total number of offsets, index and tag bits.

Off Set Bits =

Index Bits =

Tag Bits =

10. Assume a **4-way set associative** cache of size **16KB**, each block having four 32-bit words. The system is based on 32-bit physical address and uses byte addressing. Find the total number of offsets, index and tag bits.

Off Set Bits =

Index Bits =

Tag Bits =

## COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 4

---

11. Show final cache where block size =2, N = 4

|    |     |
|----|-----|
| 20 | 7   |
| 21 | 3   |
| 22 | 27  |
| 23 | 32  |
| 24 | 101 |
| 25 | 78  |
| 26 | 59  |
| 27 | 24  |
| 28 | 56  |
| 29 | 87  |
| 30 | 36  |
| 31 | 98  |

|   | Address | Data |
|---|---------|------|
| 0 |         |      |
| 1 |         |      |
| 2 |         |      |
| 3 |         |      |

|   |         |
|---|---------|
| 1 | Read 30 |
| 2 | Read 31 |
| 3 | Read 30 |
| 4 | Read 26 |
| 5 | Read 25 |
| 6 | Read 28 |
| 7 | Read 23 |
| 8 | Read 25 |
| 9 | Read 28 |

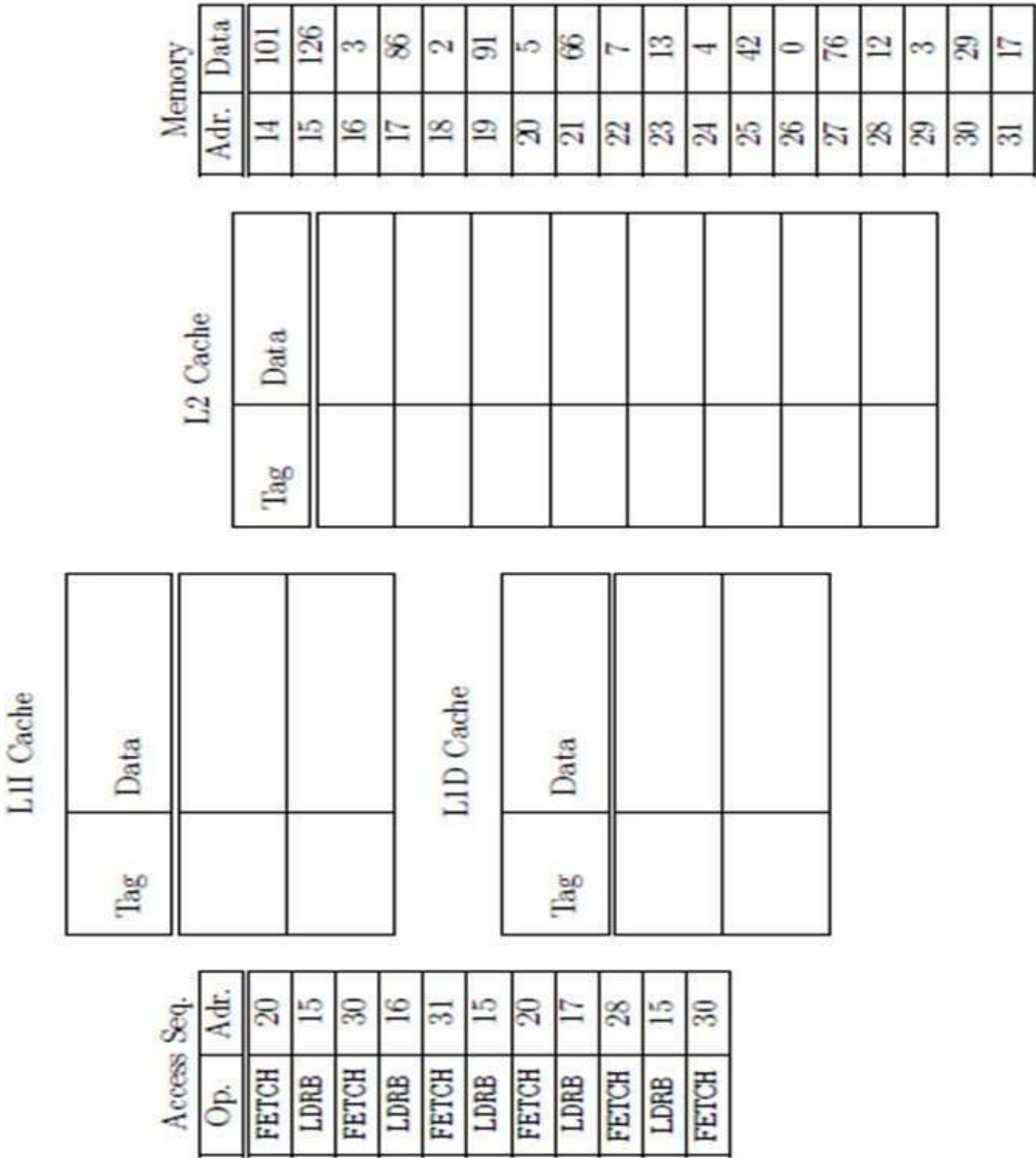
|    |         |
|----|---------|
| 10 | Read 30 |
| 11 | Read 31 |
| 12 | Read 30 |
| 13 | Read 26 |
| 14 | Read 25 |
| 15 | Read 28 |
| 16 | Read 23 |
| 17 | Read 25 |
| 18 | Read 28 |

hits = hit rate =

misses = miss rate =

12. Show the final data and tags of the two levels of cache, the given memory contents and data access sequence. Consider 5-bit memory addresses and single byte instructions. Assume that the L1 caches are direct mapped and the L2 cache is fully associative, using LRU for replacement. All caches can place a single byte of data in each block. Also assume that **addresses greater than 19 contain instructions whereas addresses smaller than 19 contain data.**

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE  
ASSIGNMENT 4



L1(Instruction) miss rate =

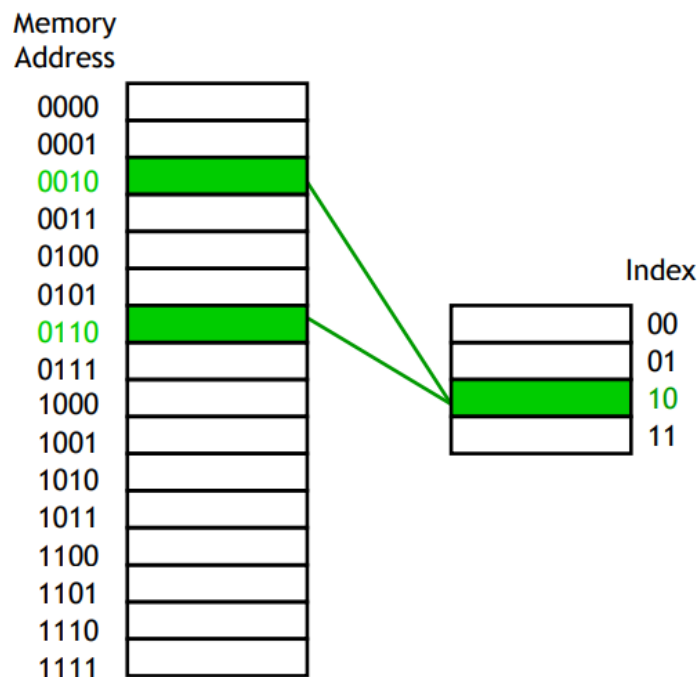
L1(Data) miss rate =

L2 miss rate =

## COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 4

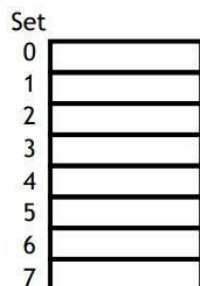
---

13. If we have a direct mapped cache and memory as below, what happens if a program uses addresses 2, 6, 2, 6, 2, ?

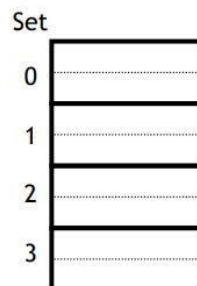


14. Considering the previous question memory and addresses, what happened if we have a fully associative cache?

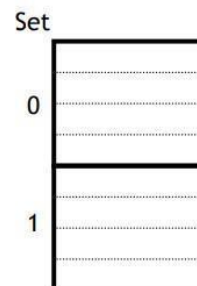
1-way associativity  
8 sets, 1 block each



2-way associativity  
4 sets, 2 blocks each



4-way associativity  
2 sets, 4 blocks each



**COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE  
ASSIGNMENT 4**

---

15. Considering the below information:

| Technology     | Primary Memory    | Secondary Memory    | Block      |
|----------------|-------------------|---------------------|------------|
| Cache Memory   | SRAM Cache        | DRAM Main Memory    | Cache Line |
| Virtual Memory | DRAM Main Memory  | Disk Memory         | Page       |
| Access Time    | TP (Primary Time) | TS (Secondary Time) |            |

Effective Access Time:  $T_E = h * T_P + (1 - h) * T_S$ , where  $h$  (the primary hit rate) is the fraction of memory accesses satisfied by the primary memory;  $0.0 \leq h \leq 1.0$ .

This formula does extend to multi-level caches. For example, a two-level cache has  $T_E = h_1 * T_1 + (1 - h_1) * h_2 * T_2 + (1 - h_1) * (1 - h_2) * T_S$ .

- Suppose a single cache fronting a main memory, which has 80 nanosecond access time. Suppose the cache memory has access time 10 nanoseconds.

(a) If the hit rate is 90%, then  $T_E = ?$

(b) If the hit rate is 99%, then  $T_E = ?$

(c) Suppose a L1 cache with  $T_1 = 4$  nanoseconds and  $h_1 = ?$

(d) Suppose a L2 cache with  $T_2 = 10$  nanoseconds and  $h_2 = ?$

(e) Suppose a main memory with  $T_S = 80.0$  then  $T_E = ?$

Q16: Write about the design architecture of the caches used in the following devices and also write a note on why this design architecture is suitable for this device.

1- .386

2- Your Mobile Phone

3- Your Laptop

Q17: Design Cache for direct Mapping? If your Main Memory address size is 7-bits?

**Note: Tag is 3-bit, Index/set address is 4 bits, no offset bits**

Q18: Design Cache for Set Associative? If your Main Memory address size is 6-bits?

**Note: Tag is 3-bits, Index/line is 1 bit, offset is 2-bits, Associativity=4?**

Q19: Fill in the Direct-mapped cache for the address given below. Main memory is given on board. Calculate Hit and Miss Rate.

|                |     |     |     |     |     |     |     |     |     |     |     |     |     |
|----------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Address        | 30h | 3FH | 32H | 38H | 39H | 33H | 3EH | 31H | 10H | 1EH | 11H | 38H | 33H |
| Binary Address |     |     |     |     |     |     |     |     |     |     |     |     |     |
| Hit/Miss       |     |     |     |     |     |     |     |     |     |     |     |     |     |

| Tag |     | 0 | 1 | V |
|-----|-----|---|---|---|
|     | 000 |   |   |   |
|     | 001 |   |   |   |
|     | 010 |   |   |   |
|     | 011 |   |   |   |
|     | 100 |   |   |   |
|     | 101 |   |   |   |
|     | 110 |   |   |   |
|     | 111 |   |   |   |

Hit Rate=

Miss Rate=

Capacity=

bytes=

## Q20: Cache Memory Simulator

**Objective:** Learn how a cache memory works, including concepts such as associativity, hits, misses, and eviction policies.

### Instructions:

1. Download the provided Python file(cache\_simulator.py) and run it using PyQt5.
2. Experiment with different cache and memory configurations.
3. Record your observations for the scenarios provided.
4. Answer the following questions based on your experiments.

### Tasks:

- Task 1: Configure the cache and memory with different parameters and observe the number of cache sets.
- Task 2: Access memory addresses using "Read" and "Write" operations. Record hits and misses.
- Task 3: Experiment with low associativity and explain eviction behavior.
- Task 4: Answer these questions:
  - Q1: What is the relationship between block size and the number of cache sets?
  - Q2: How does increasing associativity affect the cache miss rate?
- Task 5: Convert the **access\_cache** function to Assembly code(MASM 8086):
  - Calculate block number and set index using division and modulo.
  - Check cache sets for a matching tag (loop).
  - Handle hits and misses using assembly logic.

## Installation Guide for Question 20:

### Step 1: Install Python

1. Download Python:
  - Go to the official Python website: <https://www.python.org/downloads/>
  - Download the latest version of Python for your operating system (Windows/Mac/Linux).
2. Install Python:
  - Run the downloaded installer.
  - Important: During installation, check the box "Add Python to PATH" (this will make Python accessible from the command line).
  - Click on "Customize Installation" (optional) to choose additional components, then click "Install Now."
3. Verify Installation:
  - Open a terminal (Command Prompt, Terminal, or PowerShell).
  - Type: `python --version`
  - You should see the Python version installed. If not, ensure Python is added to your PATH.

### Step 2: Install PyQt5

1. Open a Terminal:
  - On Windows: Open Command Prompt or PowerShell.
  - On Mac/Linux: Open Terminal.
2. Install PyQt5 Using pip:

Type the following command and press Enter:

bash

Copy code

```
pip install pyqt5
```

- This command installs the PyQt5 library, which is used for creating graphical user interfaces (GUIs) in Python.

3. Install PyQt5 Tools (Optional):

If you want access to the Qt Designer tool (for visually designing GUIs), run:

bash

Copy code

```
pip install pyqt5-tools
```

- After installation, the Qt Designer executable can be found in the installed Python directory under `Lib\site-packages\qt5_applications\Qt\bin`.

4. Verify PyQt5 Installation:

Run the following command to check if PyQt5 is installed correctly:

bash

Copy code

```
python -c "from PyQt5.QtWidgets import QApplication; print('PyQt5 is installed successfully')"
```

- If there's no error, PyQt5 is installed properly.

### **Step 3: Running the Cache Simulator Code**

1. Copy the Code:
  - Save the provided Python code for the Cache Simulator into a file named **cache\_simulator.py**.
2. Run the Code:
  - Open a terminal in the folder where the **cache\_simulator.py** file is saved.

Run the following command:

bash

Copy code

```
python cache_simulator.py
```

- The Cache Simulator window should appear.

### **Final Submission:**

- The final submission must include the following files:
  - **Scanned Handwritten PDF**: Your scanned handwritten assignment in PDF format.
  - **Source Program for Q20**: The source code for Task 5 of Q20 in the appropriate programming language (e.g., MASM 8086 Assembly).
  - **Observation Report (Soft Copy)**: A report detailing your observations, conclusions, and any additional information related to Q20. The report should be in a readable format (e.g., .doc, .pdf, or .txt).